



INFORMATION
TECHNOLOGY
UNIVERSITY

Name:

Muhammad Abu Bakar

Roll Number:

BSCS25109

Degree:

BS Computer Science (Semester I)

Section:

A

Subject:

Programming Fundamental

PROJECT:

01

Title:

BANK ATM MANAGEMENT SYSTEM

CODE EXPLANATION

I made ATM and bank management system program by using

- Algorithmic thinking: searching, sorting, validation, edge-case handling.
- Language: C++17 or later
- Modular decomposition: clear separation of concerns into small functions/modules (even though there are no classes).
- File handling: robust load/save, append-only logs, simple crash safety.
- Data modeling with arrays: parallel arrays and index management.
- User experience in the console: clear menus

I started the program where all account details are loaded into arrays using the loadAccounts() function from accounts.txt file. These arrays stored the account number, Personal Identification Number, balance, status ,number of failed login attempts, lock status, and whether the account needs admin approval. These were used as global variable so we could use it anywhere in the program.

```
long long accountNumbers[maxAccounts];  
  
char pins[maxAccounts][6];  
  
long long balances[maxAccounts];  
  
char status[maxAccounts];  
  
int failedAttempts[maxAccounts];  
  
int locked[maxAccounts];  
  
int needsApproval[maxAccounts];
```

When I performed any operation in bank ATM management system, it is saved the values and wrote them back to the file through the saveAccounts() function, it confirmed permanent storage of all changes. All operations i.e., deposits, withdrawals, and transfers are recorded in transactions.txt using the recordTransactions() function, which automatically assigns a new transaction ID by reading the last entry from the file.

```
cout << "<=====>" << endl;
```

```

cout << "          BANK ATM SYSTEM          " << endl;
cout << "<=====>" << endl;
cout << "1. Customer Login" << endl;
cout << "2. Manager Login" << endl;
cout << "3. Exit" << endl << endl;
cout << "<=====>" << endl << endl;
cout << "Enter choice: ";
cin >> ch;

```

The main menu gave three choices:

1. Customer login
2. Manager login
3. Exit.

For customer mode, the system provided a secure login using masked PIN input through `_getch()`. It checked account status, lock status, and login attempts. After three incorrect PIN attempts, the account is locked automatically. When the customers are login successfully then the Customer Menu will show, which includes deposit, withdraw, check balance, transfer, change PIN, mini-statement and logout

```

cout << "<=====>" << endl;
cout << "          CUSTOMER MENU          " << endl;
cout << "<=====>" << endl;
cout << "1. Deposit" << endl;
cout << "2. Withdraw" << endl;
cout << "3. Check Balance" << endl;
cout << "4. Transfer" << endl;
cout << "5. Change PIN" << endl;
cout << "6. Mini Statement" << endl;

```

```

cout << "7. Logout" << endl<<endl;

cout << "<----->" << endl;

cout << "Enter choice: ";

cin >> c;

```

Each function updates both the internal arrays and the respective files. The mini-statement reads the last few transactions from transactions.txt, filters them according to the logged-in account, and displayed them with proper labels such as Deposit, Withdraw, TransferIn, and TransferOut.

For manager mode, the Manager Login used a simple reversible obfuscation, where the actual password in admin.txt is **BSCS25109** that is stored after shifting ASCII values by +5 which is **GXXH7:65>**. During login, it shifts them back to compare with user input. When manager logged in, he could create new accounts, which were assigned the next available account number that is unique in nature. The manager can also close accounts, activate or deactivate accounts, unlock locked accounts, search for a specific account, display all accounts in ascending or descending order based on balance, view summary reports, and approve new accounts.

I used sorting like basic bubble sort to reorder accounts by balance. The summary report counts active, deactivated, closed, and locked accounts while also computing the total balance stored in the bank.

```

cout << "<----->" << endl;

cout << "          MANAGER MENU          " << endl;

cout << "<----->" << endl;

cout << "1. Open New Account" << endl;

cout << "2. Close Account" << endl;

cout << "3. Deactivate/Activate" << endl;

cout << "4. Unlock Account" << endl;

```

```

cout << "5. Search Account" << endl;

cout << "6. Display All" << endl;

cout << "7. Summary Report" << endl;

cout << "8. Approve Accounts" << endl;

cout << "9. Back" << endl<<endl;

cout << "<=====>" << endl<<endl;

cout << "Enter choice: ";

cin >> ch;

```

I used some header files like:

- #include <iostream> :
For input and output (cout, cin)
- #include <iomanip> :
For formatting output (setw,)
- #include <fstream> :
For file handling (reading/writing files)
- #include <cstdlib> :
For general utilities (system(cls))
- #include <conio.h>:
For _getch() and keyboard input without echo
- #include <windows.h>:
For Windows-specific functions (Sleep)

Throughout the execution, the system used file handling, loops, secure input, conditional checks, and messages to simulate a real-world ATM system. Upon exiting, a farewell ASCII art message is displayed, thanking the user for using the system.

```

cout << "<=====>" << endl;
cout << "    Thank you for using BANK ATM SYSTEM    " << endl;
cout << "            Your money is safe                " << endl;
cout << "                    /\_/\                    " << endl;
cout << "                    ( o.o )                  " << endl;

```

```
cout << "                > ^ <                " << endl;
cout << "                Have a secure and pleasant day                " << endl;
cout << "<=====>" << endl;
```

TEXT FILE

I created three files:

1. accounts.txt
2. transaction.txt
3. admin.txt

accounts.txt

This file stores all customer account information like account number, pin, balance, status, failed attempt, lock details and need approval.

account.txt sample data:

accountNumber	pin	balance	status	failedAttempts	locked	needsApproval
10001	1122	5000	A	0	0	0
10002	3344	2000	A	0	0	0
10003	5566	0	A	0	0	1
10004	7788	15000	D	0	0	0

transaction.txt

This file records every transaction such as deposit, withdraw, and transfer with a unique transaction ID.

transactions.txt sample data:

TransactionID	Type	SourceAccount	DestinationAccount	Amount
1	1	10001	-1	2000
2	4	10001	-1	1000
3	2	10001	10002	500

4

3

10002

10001

500

admin.txt

This file saves the admin password in an obfuscated or shifted form for login security

admin.txt sample data:

GXHX7:65>

->BSCS25109